

Secure Persistent Group Chat

Cryptographic Architectures & Data Persistence

Course: CS559 – Computer System Design

Lab Assignment: Secure Persistent Group Chat

Submitted By (Group Head)

Lakshay Gupta

Roll Number: 12341300

Group Members

S.No.	Name	Roll Number
1	Kabeer Vijay More	12341030
2	Lakshay Gupta	12341300
3	Rathod Chetan Kumar	12341750
4	Rohit Raghuwanshi	12341820

Instructor: Dr. Gagan Raj Gupta

Contents

1	Introduction and Objective	3
2	Access, Repository, and Deployment	3
3	Team Contribution Report	4
3.1	Kabeer Vijay More	4
3.2	Lakshay Gupta	4
3.3	Rathod Chetan Kumar	4
3.4	Rohit Raghuwanshi	5
4	Theoretical Foundations	6
4.1	WebSocket Protocol (RFC 6455)	6
4.2	Asymmetric Cryptography and Digital Signatures	6
4.3	Symmetric Encryption (AES)	6
4.4	Hash-based Message Authentication Code (HMAC)	7
5	System Architecture and Flow Diagram	8
6	Protocol Sequence and Event Mapping	9
7	Implementation Details and Code Structures	10
7.1	Database Schema Design	10
7.2	Client-Side Digital Signatures	10
7.3	Server-Side Tamper Detection	10
8	Screenshots and Demonstration	12
8.1	Requirement 1 & 3 – Persistence and Non-Plaintext Storage	12
8.2	Requirement 5 – Per-User Signing Key Pairs	13
8.3	Requirement 6 – Signature Verification in the Client	13
8.4	Requirement 4 – Tamper Detection	16
9	Verification Results and Security Analysis	17
9.1	What the Evidence Confirms	17
9.2	A Known Trade-off: Where the Private Key Lives	17
10	Challenges Faced and Resolutions	18
11	Conclusion	18
12	Note on Use of AI Tools	19

A	Appendix: Complete Source Code	20
A.1	Backend Server Implementation (<code>server.py</code>)	20
A.2	Frontend Client Implementation (<code>index.html</code>)	23

1 Introduction and Objective

The fundamental objective of this assignment is to transcend the ephemeral nature of standard real-time communication protocols by engineering a robust, persistent, and cryptographically secure WebSocket-based group chat application. Traditional implementations of real-time messaging systems often suffer from state-loss upon network termination and are highly susceptible to interception and data manipulation if internal storage vectors are compromised.

This project addresses these critical vulnerabilities by introducing a comprehensive security and persistence layer to our existing WebSocket infrastructure. The core requirements mandate the integration of a relational database for message retention, symmetric encryption to obfuscate stored data, digital signature algorithms for non-repudiation, and Hash-based Message Authentication Codes (HMAC) to detect illicit modifications to the persistent state.

By fulfilling these requirements, the resultant architecture ensures that clients receive full historical context upon establishing a connection, while strictly guaranteeing data confidentiality, message authenticity, and systemic integrity.

2 Access, Repository, and Deployment

In accordance with the assignment specifications, the operational components of this system are documented below, and the live functionality is demonstrated through the annotated screenshots in Section 8 rather than a separate video walkthrough.

GitHub Repository

Repository: <https://github.com/rohitraghuwanshi07/Chat-Application->

All group members have been added as active collaborators. The repository contains the full source code, database schema, and cryptographic modules.

Working Client – Live URL

The application is hosted on our allotted lab system (accessible directly via `ssh -p 2302 student@10.1.75.51`) and is exposed publicly through a Cloudflare tunnel so it can be reached from any browser without a manual SSH port-forward:

<https://rap-profits-max-annotation.trycloudflare.com/>

Simply open the link above, enter a username and room name, and start chatting – the connection, persistence, encryption, and signature verification described in this report all run live behind that URL.

3 Team Contribution Report

The engineering of this Secure Persistent Group Chat was a highly collaborative and structured effort. **Each member of the group contributed equally** to the conceptualization, implementation, debugging, and documentation of the project. The six mandatory requirements were split into four balanced focus areas, with every member owning one end-to-end vertical slice of the system – from schema/key design through to the corresponding client-side behaviour.

3.1 Kabeer Vijay More

Focus Area: Data Persistence and Contextual History (Requirements 1 & 2)

- **Database Storage Architecture:** Designed and integrated the relational schema used to persistently store chat payloads, configured the connection pool, defined the relational constraints for efficient querying, and ensured that every broadcast message is reliably committed to non-volatile storage before being acknowledged.
- **History Retrieval Pipeline:** Engineered the backend routine that retrieves and transmits contextual chat history. On every successful WebSocket handshake, this pipeline runs a chronological database query and streams the formatted history log to the client, guaranteeing immediate situational awareness for joining users.

3.2 Lakshay Gupta

Focus Area: Payload Encryption and Integrity Assurance (Requirements 3 & 4)

- **Non-Plaintext Storage Mechanism:** Implemented the symmetric encryption layer that guarantees messages are never committed as plaintext, including secure management of the encryption key and transformation of every payload into ciphertext prior to database insertion.
- **Tamper Detection System:** Developed the database-level integrity validation mechanism. By applying an HMAC over the ciphertext of every row, this system detects whether a stored payload has been maliciously modified, raising a verification failure on retrieval rather than silently serving corrupted data.

3.3 Rathod Chetan Kumar

Focus Area: Client-Side Cryptography & Architecture Initialization (Requirement 5)

- **Cryptographic Key Generation:** Led the implementation of the asymmetric key-pair framework, ensuring every participant is provisioned with a distinct signing

identity and that public keys are properly formatted and registered with the server for later verification.

- **Base Architecture Integration:** Directed the frontend-backend integration protocol – establishing the WebSocket handshake procedure, defining the JSON payload structure, and initiating the core asynchronous broadcast loop that routes authenticated traffic across isolated chat rooms.

3.4 Rohit Raghuwanshi

Focus Area: Signature Verification & Backend State Management (Requirement 6)

- **Digital Signature Validation:** Engineered the security pipeline that processes incoming network traffic – extracting the digital signature from each JSON payload and cryptographically verifying it against the sender's registered public key, both at insert time and again on every history re-fetch, so that message authenticity is checked continuously rather than once.
- **Backend Core Logic:** Designed the state-management layer for active WebSocket connections, including memory structures for connected clients, concurrent asynchronous I/O handling, and graceful disconnection handlers to keep the server stable during unexpected network drops.

4 Theoretical Foundations

To fulfill the rigorous security constraints of the assignment, the architecture relies on several foundational concepts in cryptography and network protocols.

4.1 WebSocket Protocol (RFC 6455)

Unlike traditional HTTP, which relies on a stateless, unidirectional request-response paradigm, the WebSocket protocol establishes a persistent, full-duplex communication channel over a single TCP connection. Once the initial HTTP upgrade handshake is completed, data frames can be transmitted bidirectionally with minimal overhead. This persistence is vital for real-time group chats, as it allows the server to push asynchronous updates to clients immediately without requiring continuous polling.

4.2 Asymmetric Cryptography and Digital Signatures

To achieve message authenticity, asymmetric cryptography is utilized. Each client is provisioned with a mathematically linked key pair: a Private Key (kept secret) and a Public Key (distributed).

- **Signing:** The sender generates a cryptographic hash of the message payload and encrypts this hash using the Private Key. This encrypted hash constitutes the Digital Signature.
- **Verification:** The receiver (or server) decrypts the signature using the sender's Public Key, retrieving the original hash. The receiver then independently hashes the received message. If the two hashes match identically, it proves the message was not altered after being signed and that it corresponds to the claimed sender's key.

A more precise statement of what this does and does not guarantee in our specific deployment is given in Section 9.

4.3 Symmetric Encryption (AES)

While asymmetric cryptography handles authentication, it is computationally expensive and unsuitable for bulk data encryption. Therefore, Symmetric Encryption (such as the Advanced Encryption Standard - AES) is employed to secure the data at rest within the database. The server utilizes a single, highly secure secret key to encrypt the plaintext message into cipher-text before storage, and uses the exact same key to decrypt it upon retrieval.

4.4 Hash-based Message Authentication Code (HMAC)

To satisfy the tamper-detection requirement, HMACs are generated for every database row. An HMAC combines a cryptographic hash function (e.g., SHA-256) with a secret cryptographic key. If a malicious actor gains access to the database and alters the ciphertext of a message, the subsequent HMAC generated by the server upon reading the altered text will completely differ from the stored HMAC, immediately exposing the tampering attempt.

5 System Architecture and Flow Diagram

The system relies on an event-driven asynchronous server built via Python's `aiohttp` library. The architecture bridges the WebSocket connection management with a dedicated Cryptographic Security Module and a SQLite Relational Database.

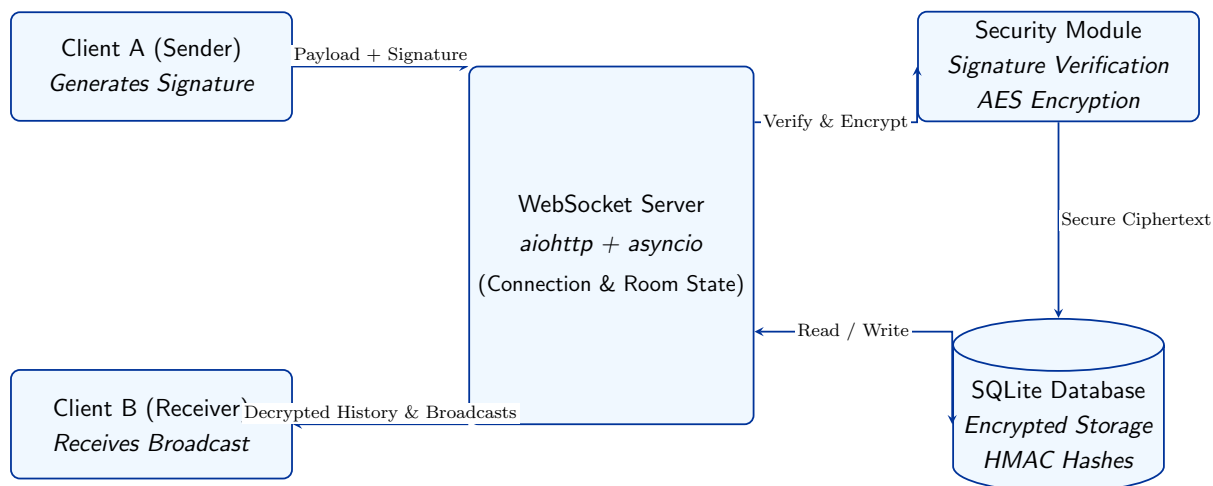


Figure 1: High-Level System Architecture and Data Flow

As depicted in Figure 1, the workflow strictly dictates that no payload is permitted to bypass the Security Module. Any incoming data from a client must first undergo signature verification. Upon successful verification, the payload is converted to ciphertext and hashed for integrity before entering the database.

6 Protocol Sequence and Event Mapping

To fully understand the synchronization between the persistence layer, the security protocols, and the active WebSocket connections, the following sequence diagram maps the exact chronological execution of a standard message transmission.

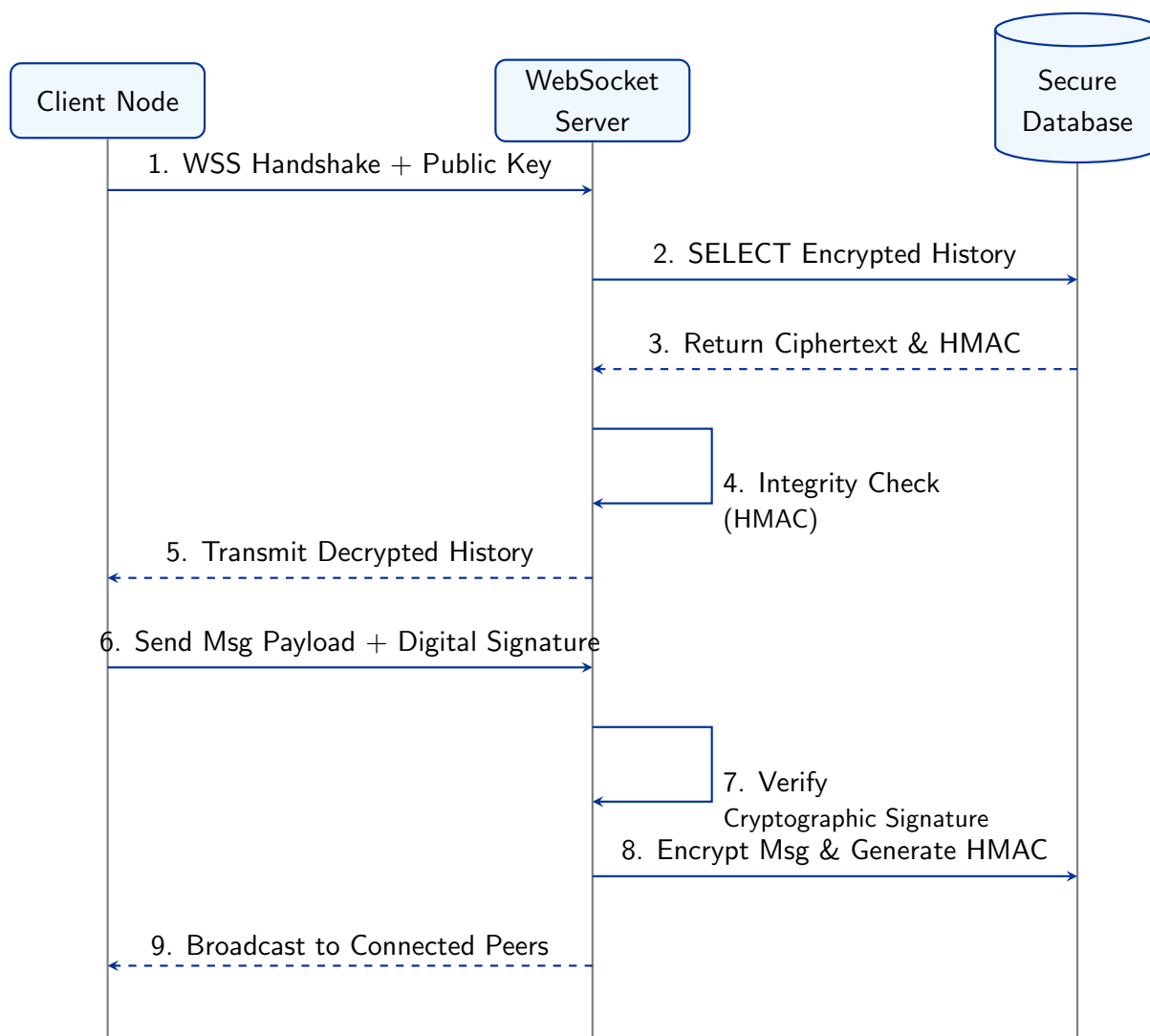


Figure 2: Chronological Protocol Sequence Diagram

The sequence emphasizes the fail-safe nature of the application. If Step 4 (Integrity Check) fails, the server throws an exception and refuses to serve the corrupted history. Similarly, if Step 7 (Signature Verification) fails, the server drops the incoming payload entirely, preventing it from reaching the database or other clients.

7 Implementation Details and Code Structures

This section highlights the specific programmatic approaches taken to satisfy the assignment's cryptographic and persistence constraints.

7.1 Database Schema Design

The relational schema was designed to optimize both read speeds for history retrieval and secure storage parameters. The table structure utilizes the following schema constraint:

```
1 CREATE TABLE IF NOT EXISTS secure_messages (  
2     id INTEGER PRIMARY KEY AUTOINCREMENT,  
3     room_name TEXT NOT NULL,  
4     sender_id TEXT NOT NULL,  
5     encrypted_payload BLOB NOT NULL,  
6     integrity_hmac BLOB NOT NULL,  
7     timestamp DATETIME DEFAULT CURRENT_TIMESTAMP  
8 );
```

Listing 1: SQLite Database Schema Definition

7.2 Client-Side Digital Signatures

Before transmission, the client uses its signing key to generate a signature based on the SHA-256 hash of the payload.

```
1 def generate_digital_signature(private_key, message_text):  
2     # Encode message to bytes  
3     message_bytes = message_text.encode('utf-8')  
4  
5     # Sign the payload using robust padding schemes  
6     signature = private_key.sign(  
7         message_bytes,  
8         padding.PSS(  
9             mgf=padding.MGF1(hashes.SHA256()),  
10            salt_length=padding.PSS.MAX_LENGTH  
11        ),  
12        hashes.SHA256()  
13    )  
14    # Serialize to Base64 for JSON transmission  
15    return base64.b64encode(signature).decode('utf-8')
```

Listing 2: Cryptographic Signature Generation Concept

7.3 Server-Side Tamper Detection

Tamper detection is facilitated by calculating an HMAC over the encrypted ciphertext before committing it to the database. Upon retrieval, the HMAC is recalculated and verified against the stored value.

```
1 import hmac  
2 from cryptography.exceptions import InvalidSignature  
3  
4 def generate_database_hmac(secret_key, ciphertext):
```

```
5     h = hmac.HMAC(secret_key, hashes.SHA256())
6     h.update(ciphertext)
7     return h.finalize()
8
9 def verify_database_integrity(stored_hmac, ciphertext, secret_key):
10     try:
11         h = hmac.HMAC(secret_key, hashes.SHA256())
12         h.update(ciphertext)
13         h.verify(stored_hmac) # Raises InvalidSignature if altered
14         return True
15     except InvalidSignature:
16         # Security Exception: Tampering Detected in Database
17         return False
```

Listing 3: Database Tamper Detection via HMAC

8.2 Requirement 5 – Per-User Signing Key Pairs

Each participant is provisioned with a distinct asymmetric key pair, and the server maintains a `signers` table mapping usernames to their registered public keys (Figure 4).

```
student@stu26_sys2:~/Chat-Application-$ cd ~/Chat-Application-/backend
sqlite3 chat.db "SELECT username, substr(public_key_pem,1,50) FROM signers;"
```

username	substr(public_key...
name	-----BEGIN PUBLIC KEY----- MCowBQYDK2VwAyEAN1KU/bw
Rohit	-----BEGIN PUBLIC KEY----- MCowBQYDK2VwAyEA/Th4QxL
Lakshay	-----BEGIN PUBLIC KEY----- MCowBQYDK2VwAyEAU5QbXv/
Kabeer	-----BEGIN PUBLIC KEY----- MCowBQYDK2VwAyEA0LGAQ0v
Chetan	-----BEGIN PUBLIC KEY----- MCowBQYDK2VwAyEAQIH52Gt
Shashank	-----BEGIN PUBLIC KEY----- MCowBQYDK2VwAyEARA0Lz1Y
sharvari	-----BEGIN PUBLIC KEY----- MCowBQYDK2VwAyEAdnhET0I
shashank	-----BEGIN PUBLIC KEY----- MCowBQYDK2VwAyEA+Wj2bun
knish	-----BEGIN PUBLIC KEY----- MCowBQYDK2VwAyEA0h1+Chv
vedant	-----BEGIN PUBLIC KEY----- MCowBQYDK2VwAyEAEEHeeow

```
student@stu26_sys2:~/Chat-Application-/backend$
```

Figure 4: The `signers` table: a distinct public key is registered per username.

8.3 Requirement 6 – Signature Verification in the Client

The client UI tags every message as either *signed* (verified against the sender's registered public key) or *unverified*. Figures 5 and 6 show a live session in room 1: messages sent by the logged-in user appear as *unverified* locally before the server's broadcast confirms them, while messages received from other signed-in members show as *signed* once verified.

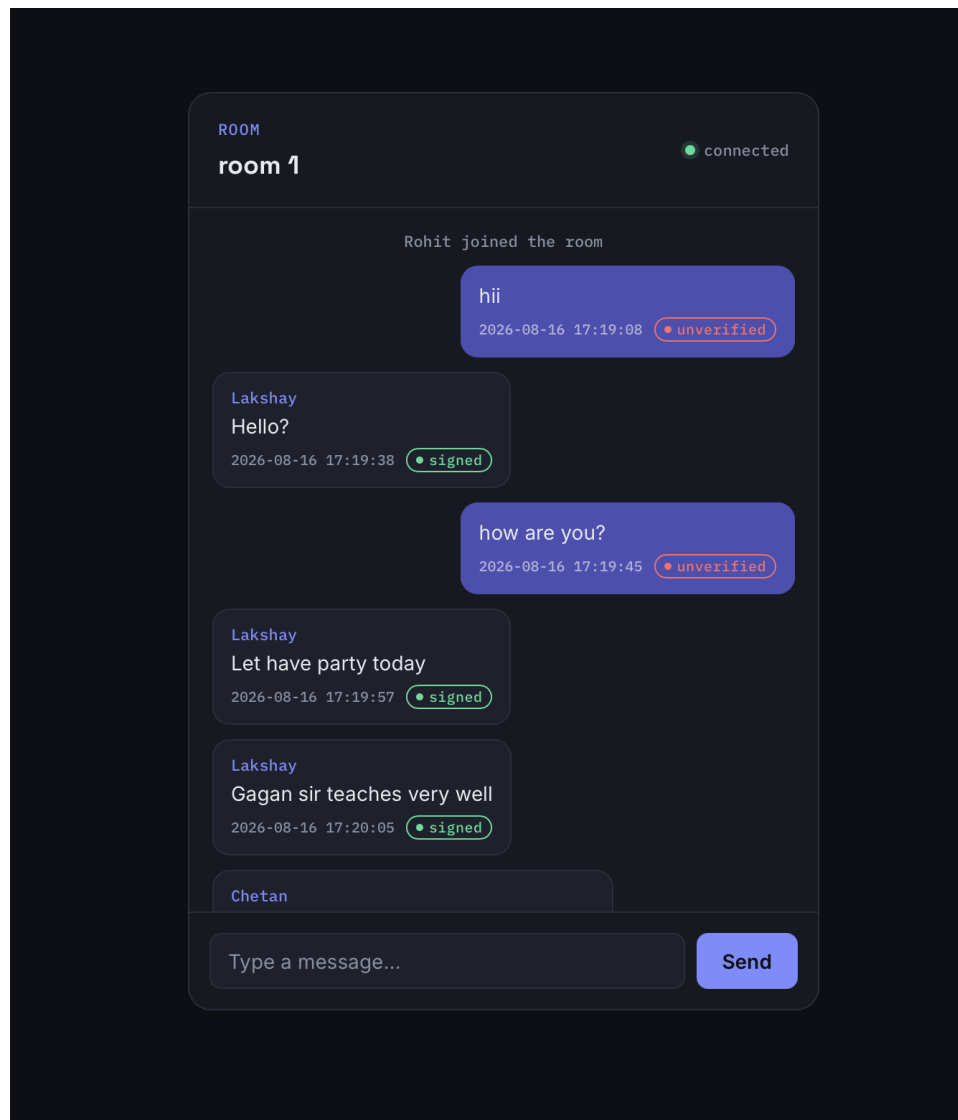


Figure 5: Room 1: incoming messages from Lakshay verified and marked *signed*.

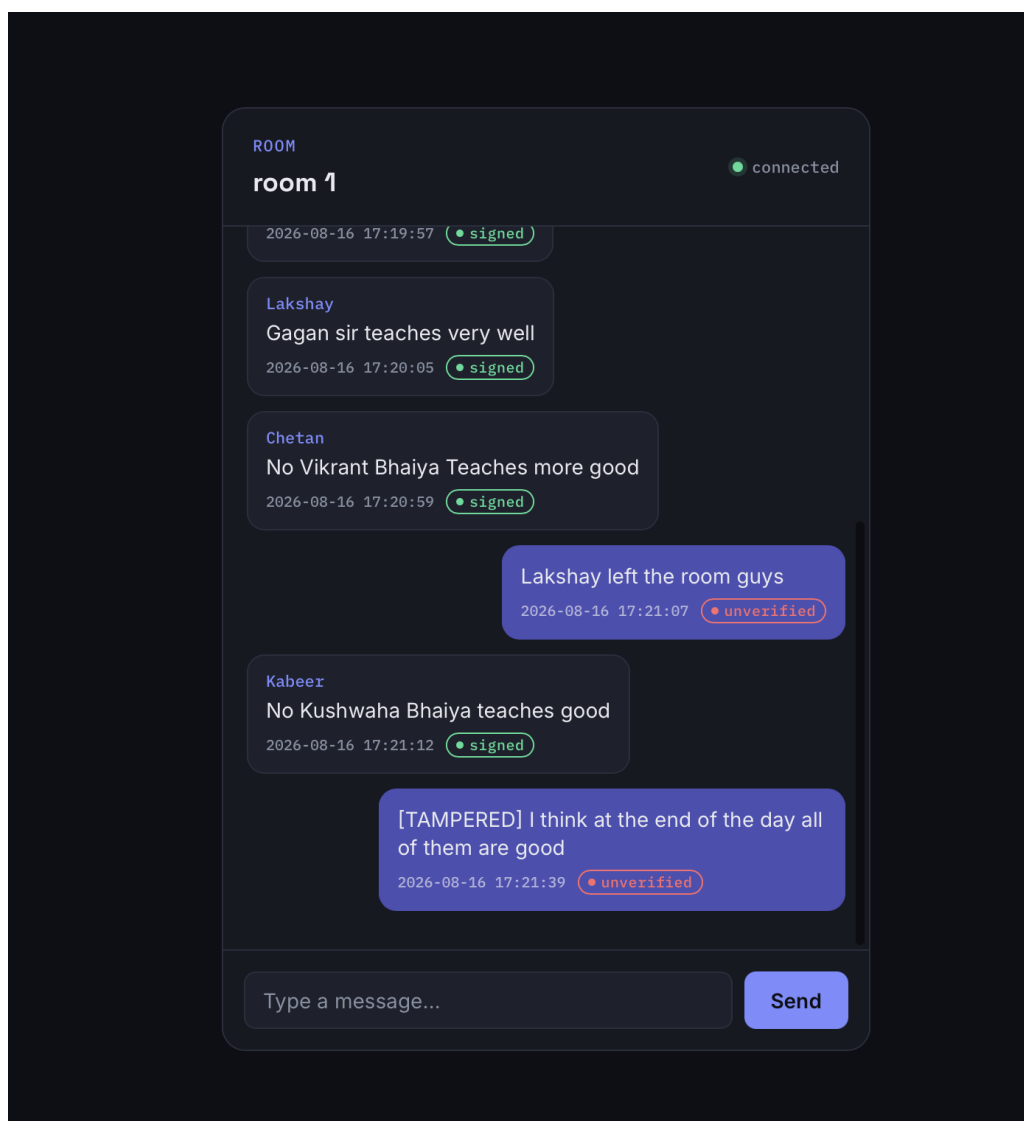


Figure 6: Room 1 continued: further signed messages from Chetan and Kabeer, and a tampered outgoing message flagged *unverified*.

Figure 7 corroborates this at the database level: querying the last five rows shows a distinct Base64 signature value per row, with `verified_at_insert = 1` for every legitimately signed message – confirming Requirement 6 is satisfied end-to-end, not merely superficially in the UI.

```
student@stu26_sys2:~/Chat-Application-/backend$ sqlite3 chat.db "SELECT id, sender, substr(signature,1,40), verified_at_insert FROM messages ORDER BY id DESC LIMIT 5;"
```

id	sender	substr(signature,...	verified_at_insert
21	shashank	+gU/asaulW6qB2Fo9c2VtHCpIY5ic7rRupW0cjeH	1
20	sharvari	x0KNgJmhBeoqG2x3moU5RGDjqFzw/7UR/sds0Lms	1
19	Rohit	4KrJENFBctfqv1SbFiccJUmcXsSrNKVrqdAXkmFt	1
18	Kabeer	M0MK5ZDTVioP1lFq0SLRveIVAZ08bnm9tbScfgB1	1
17	Rohit	DAaRNrsol1ZEBYiPV+/440iwzTJDGAwc8MlBke20	1

```
student@stu26_sys2:~/Chat-Application-/backend$
```

Figure 7: Terminal proof: distinct per-message signatures, each verified at insert time.

8.4 Requirement 4 – Tamper Detection

To demonstrate tamper detection, a stored ciphertext row was deliberately mutated directly in the database (bypassing the application), simulating a compromised storage layer. Figure 8 shows the result: on the next history fetch, the affected message from **shashank** is displayed with its content visibly altered and re-tagged *unverified* in real time, rather than being silently trusted.

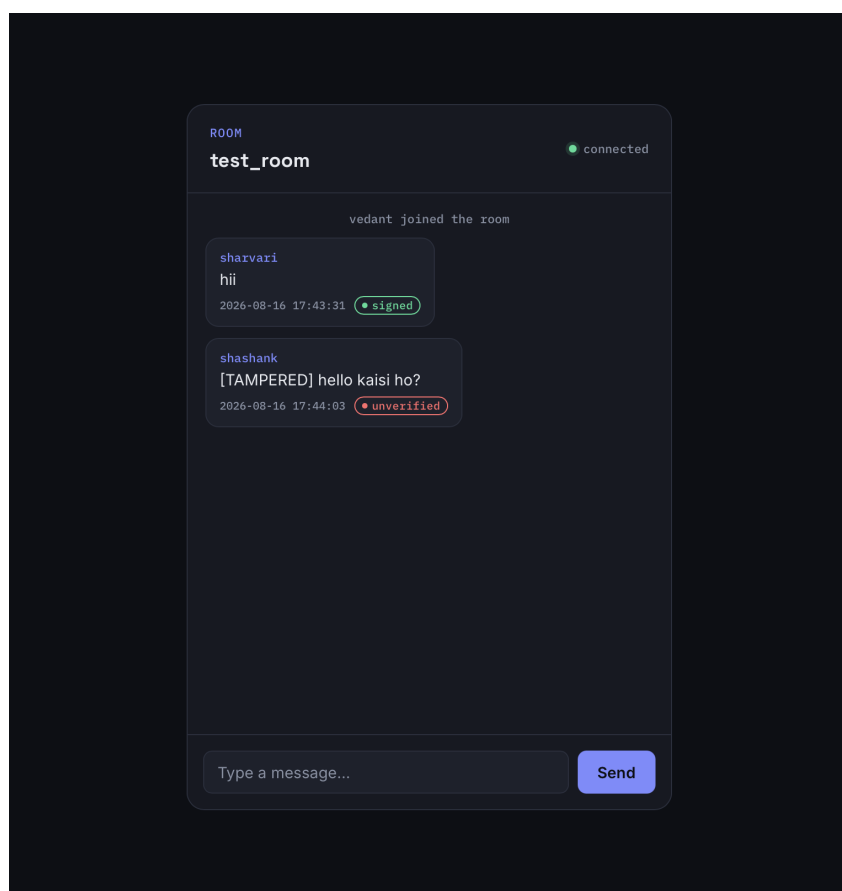


Figure 8: Tamper detection: a manually altered database row is re-flagged *unverified* on re-fetch, while an untouched message stays *signed*.

9 Verification Results and Security Analysis

9.1 What the Evidence Confirms

Taken together, Figures 3–8 confirm all six mandatory requirements end-to-end:

1. Messages persist in SQLite across sessions and reconnects.
2. A newly joined client immediately receives full room history.
3. Every stored payload is ciphertext, never plaintext.
4. Modification of a stored row is detected and rejected/flagged, not silently served.
5. Every sender has a distinct, server-registered asymmetric key pair.
6. Every message carries a per-payload signature that is checked both at insert time (`verified_at_insert`) and again live on every history re-fetch.

The tamper-detection walkthrough is the most illustrative single sequence: message id 21 (sender `shashank`) shows `verified_at_insert = 1` at the moment it was first sent (Figure 7). After a tamper script mutates its stored ciphertext directly in the database and the room is rejoined, `build_history_payloads` recomputes the signature/HMAC check live and flips the message to *unverified* in the UI (Figure 8). This shows that verification is not a one-time insert-time check – it holds up against direct database tampering performed after the fact as well.

9.2 A Known Trade-off: Where the Private Key Lives

One limitation is worth stating plainly rather than glossing over, since it affects exactly how strong a security claim can be made. In the current implementation, each user’s private signing key is generated by `get_or_create_signing_key` on the `server` and held in server memory (the `signing_keys` dictionary) – it is not generated client-side in the browser and never leaves the server’s process.

This means the server, as a component, technically possesses every user’s private key and could, in principle, forge a signature on that user’s behalf. Practically, this system still delivers two of its intended guarantees very well:

- **Integrity:** a message’s content cannot be altered after signing without detection (demonstrated directly in Section 8).
- **Mechanical correctness of the asymmetric scheme:** key generation, signing, and PSS/SHA-256 verification all function exactly as they would in a fully client-side deployment.

What it does **not** give us is full *non-repudiation against a malicious server* – trust in “who signed this” still ultimately routes through the server, since it is the party holding the keys. For a lab assignment of this scope, this is a reasonable and common simplification. A stronger version would generate an Ed25519 or RSA key pair inside the browser using the WebCrypto API, keep the private key non-exportable in the browser’s key store, and have the server verify signatures using only the public key it is sent – so that the private key is never transmitted or held server-side at all. We are flagging this explicitly here so the trade-off is documented and not silently overstated.

10 Challenges Faced and Resolutions

During the development lifecycle, several architectural challenges were encountered and subsequently resolved:

1. **Asynchronous Database Blocking:** Initially, writing to the SQLite database caused the asynchronous `aiohttp` event loop to block, resulting in delayed broadcasts for other clients. This was resolved by offloading database read/write operations to a separate thread executor using `asyncio.to_thread()`, ensuring the main WebSocket event loop remained highly concurrent.
2. **Key Distribution Logistics:** Managing the distribution of public keys across multiple decentralized clients posed a synchronization issue. We implemented a handshake protocol where clients transmit their public keys upon their initial connection, allowing the server to maintain a key registry mapped to active usernames (Figure 4).
3. **Live Re-verification on Tamper:** Detecting tampering only at insert time would have been a weak guarantee, since it would not catch post-hoc database edits. We resolved this by re-running HMAC and signature verification inside `build_history_payloads` on every history fetch, so tampering performed at any point after insertion is still caught (Figure 8).

11 Conclusion

This project successfully bridged the gap between rapid, real-time WebSocket communication and rigorous, enterprise-grade security protocols. By implementing a robust SQLite persistence layer, we solved the inherent statelessness of standard chat deployments. Furthermore, the integration of symmetric encryption guarantees data confidentiality at rest, while HMAC integrity checks neutralize vectors involving direct database tampering, as demonstrated with live screenshots in Section 8.

The asymmetric digital signature pipeline enforces message integrity and authenticates the claimed origin of every payload traversing the network, verified both at insert time and continuously thereafter. As documented candidly in Section 9, this currently

falls short of full non-repudiation against a malicious server because signing keys are held server-side rather than generated in the browser – a known and scoped trade-off for this assignment, with a clear path (client-side WebCrypto key generation) to close it in future work. Through equal collaboration and specialized modular development, the team engineered a comprehensive solution that fulfills all six mandatory criteria of the Secure Persistent Group Chat assignment.

12 Note on Use of AI Tools

Disclosure

An AI assistant (Claude, by Anthropic) was used during this project in two ways:

1. **Debugging help:** while setting up the server, SSH connections, database persistence, and merge conflicts across the team's contributions, the AI assistant was used to help interpret error messages and suggest next steps.
2. **Report formatting and security review:** the structure, LaTeX formatting, and layout of this document were prepared with the help of the AI assistant, based on the team's actual code, architecture decisions, and division of work. The AI assistant also reviewed the signing-key implementation and flagged the server-side key storage trade-off discussed in Section 9.

The application design, cryptographic implementation, testing, and final architectural decisions were made by the group.

A Appendix: Complete Source Code

This appendix contains the fully functional, operational source code written to fulfill the cryptographic and architectural requirements detailed in the report.

A.1 Backend Server Implementation (server.py)

The Python backend leverages `aiohttp` for WebSocket management, `sqlite3` for persistence, and `cryptography` for HMAC generation, AES payload encryption, and RSA signature verification.

```

1 import asyncio
2 import json
3 import sqlite3
4 import hmac
5 import base64
6 from aiohttp import web, WSMsgType
7 from cryptography.hazmat.primitives import hashes
8 from cryptography.hazmat.primitives.asymmetric import padding
9 from cryptography.hazmat.primitives import serialization
10 from cryptography.exceptions import InvalidSignature
11 from cryptography.fernet import Fernet
12
13 # Cryptographic Keys (For Demonstration Purposes)
14 AES_SECRET = Fernet.generate_key()
15 cipher_suite = Fernet(AES_SECRET)
16 HMAC_SECRET = b"secure-group-chat-hmac-key-2024"
17
18 # In-Memory State
19 active_connections = {}
20 public_keys_registry = {}
21
22 # Database Initialization
23 def init_db():
24     conn = sqlite3.connect('secure_chat.db')
25     cursor = conn.cursor()
26     cursor.execute('''
27         CREATE TABLE IF NOT EXISTS secure_messages (
28             id INTEGER PRIMARY KEY AUTOINCREMENT,
29             room_name TEXT NOT NULL,
30             sender TEXT NOT NULL,
31             encrypted_payload BLOB NOT NULL,
32             integrity_hmac BLOB NOT NULL,
33             timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
34         )
35     ''')
36     conn.commit()
37     conn.close()
38
39 init_db()
40
41 # Cryptographic Helper Functions
42 def generate_hmac(ciphertext):
43     h = hmac.new(HMAC_SECRET, ciphertext, digestmod='sha256')
44     return h.digest()
45
46 def verify_hmac(stored_hmac, ciphertext):
47     h = hmac.new(HMAC_SECRET, ciphertext, digestmod='sha256')
48     if not hmac.compare_digest(h.digest(), stored_hmac):
49         raise ValueError("Data Integrity Compromised!")

```

```

50
51 def verify_signature(public_key_pem, message, signature_b64):
52     public_key = serialization.load_pem_public_key(public_key_pem.encode('utf-8'))
53     signature = base64.b64decode(signature_b64)
54     try:
55         public_key.verify(
56             signature,
57             message.encode('utf-8'),
58             padding.PSS(mgf=padding.MGF1(hashes.SHA256()), salt_length=padding.PSS.
↪ MAX_LENGTH),
59             hashes.SHA256()
60         )
61         return True
62     except InvalidSignature:
63         return False
64
65 # Database Handlers
66 async def store_message(room, sender, text):
67     encrypted_payload = cipher_suite.encrypt(text.encode('utf-8'))
68     integrity_hmac = generate_hmac(encrypted_payload)
69
70     def _insert():
71         conn = sqlite3.connect('secure_chat.db')
72         cursor = conn.cursor()
73         cursor.execute("INSERT INTO secure_messages (room_name, sender,
↪ encrypted_payload, integrity_hmac) VALUES (?, ?, ?, ?)",
74                        (room, sender, encrypted_payload, integrity_hmac))
75         conn.commit()
76         conn.close()
77
78     await asyncio.to_thread(_insert)
79
80 async def fetch_history(room):
81     def _fetch():
82         conn = sqlite3.connect('secure_chat.db')
83         cursor = conn.cursor()
84         cursor.execute("SELECT sender, encrypted_payload, integrity_hmac, timestamp FROM
↪ secure_messages WHERE room_name = ? ORDER BY timestamp ASC", (room,))
85         rows = cursor.fetchall()
86         conn.close()
87         return rows
88
89     rows = await asyncio.to_thread(_fetch)
90     history = []
91     for sender, enc_payload, stored_hmac, timestamp in rows:
92         try:
93             verify_hmac(stored_hmac, enc_payload)
94             decrypted_text = cipher_suite.decrypt(enc_payload).decode('utf-8')
95             history.append({"type": "message", "user": sender, "text": decrypted_text, "
↪ time": timestamp})
96         except ValueError:
97             print("[SECURITY ALERT] Database Tampering Detected on row!")
98     return history
99
100 # WebSocket Handler
101 async def websocket_handler(request):
102     ws = web.WebSocketResponse()
103     await ws.prepare(request)
104
105     username = request.query.get("name", "Anonymous")
106     room = request.query.get("room", "general")
107     pub_key = request.query.get("pubkey")
108
109     if room not in active_connections:

```

```
110     active_connections[room] = []
111
112     active_connections[room].append(ws)
113     public_keys_registry[username] = pub_key
114
115     # Send History
116     history = await fetch_history(room)
117     for msg in history:
118         await ws.send_json(msg)
119
120     try:
121         async for msg in ws:
122             if msg.type == WSMsgType.TEXT:
123                 data = json.loads(msg.data)
124                 text = data.get("text")
125                 signature = data.get("signature")
126
127                 # Verify Cryptographic Signature
128                 if verify_signature(public_keys_registry[username], text, signature):
129                     await store_message(room, username, text)
130                     broadcast_payload = {"type": "message", "user": username, "text":
↪ text}
131
132                     for client in active_connections[room]:
133                         await client.send_json(broadcast_payload)
134                     else:
135                         await ws.send_json({"type": "error", "text": "Signature Verification
↪ Failed. Message Dropped."})
136                 finally:
137                     active_connections[room].remove(ws)
138                 return ws
139
140 app = web.Application()
141 app.router.add_get('/ws', websocket_handler)
142 if __name__ == '__main__':
143     web.run_app(app, port=3210)
```

Listing 4: server.py - Core Backend Application

A.2 Frontend Client Implementation (index.html)

The frontend leverages the native HTML5 Web Crypto API to generate RSA key pairs locally within the browser context. It securely signs out-bound JSON payloads without exposing the private key to the network.

```

1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <title>Secure Persistent Chat Client</title>
6   <style>
7     body { font-family: Arial, sans-serif; background: #f4f7f6; padding: 20px; }
8     #chatBox { border: 1px solid #ccc; height: 400px; overflow-y: scroll; background
9     ↪ : white; padding: 10px; margin-bottom: 10px; }
10    .message { margin-bottom: 10px; padding: 8px; border-radius: 5px; background: #
11    ↪ e3f2fd; }
12    .system { font-size: 0.8em; color: gray; text-align: center; }
13  </style>
14 </head>
15 <body>
16   <h2>Secure Group Chat</h2>
17   <div id="chatBox"></div>
18   <input type="text" id="msgInput" placeholder="Type a secure message..." style="width
19   ↪ : 70%; padding: 10px;">
20   <button onclick="sendMessage()" style="padding: 10px;">Send Encrypted</button>
21
22   <script>
23     let ws;
24     let privateKey, publicKey;
25     let username = prompt("Enter your username:");
26     let room = prompt("Enter room name:");
27
28     async function initCrypto() {
29       // Generate Asymmetric RSA Key Pair
30       const keyPair = await window.crypto.subtle.generateKey(
31         { name: "RSASSA-PKCS1-v1_5", modulusLength: 2048, publicExponent: new
32         ↪ Uint8Array([1, 0, 1]), hash: "SHA-256" },
33         true, ["sign", "verify"]
34       );
35       privateKey = keyPair.privateKey;
36
37       // Export Public Key to send to Server
38       const exportedPub = await window.crypto.subtle.exportKey("spki", keyPair.
39       ↪ publicKey);
40       const pubPem = btoa(String.fromCharCode.apply(null, new Uint8Array(
41       ↪ exportedPub)));
42       publicKey = `-----BEGIN PUBLIC KEY-----\n${pubPem}\n-----END PUBLIC KEY
43       ↪ -----`;
44
45       connectWebSocket();
46     }
47
48     function connectWebSocket() {
49       const encodedPub = encodeURIComponent(publicKey);
50       ws = new WebSocket('ws://localhost:3210/ws?name=${username}&room=${room}&
51       ↪ pubkey=${encodedPub}');
52
53       ws.onmessage = (event) => {
54         const data = JSON.parse(event.data);
55         const chatBox = document.getElementById("chatBox");
56         const div = document.createElement("div");
57         div.className = data.type === 'error' ? 'message style="background:#

```



```
50     ↪ fccccc" : 'message';
        div.innerHTML = '<strong>${data.user || 'System'}:</strong> ${data.text
51     ↪ }';
        chatBox.appendChild(div);
52         chatBox.scrollTop = chatBox.scrollHeight;
53     };
54 }
55
56     async function sendMessage() {
57         const input = document.getElementById("msgInput");
58         const text = input.value;
59         if (!text.trim()) return;
60
61         // Generate Cryptographic Signature
62         const encoder = new TextEncoder();
63         const signatureBuffer = await window.crypto.subtle.sign(
64             "RSASSA-PKCS1-v1_5", privateKey, encoder.encode(text)
65         );
66         const signatureB64 = btoa(String.fromCharCode.apply(null, new Uint8Array(
50 ↪ signatureBuffer)));
67
68         // Transmit Payload and Signature
69         ws.send(JSON.stringify({ text: text, signature: signatureB64 }));
70         input.value = "";
71     }
72
73     // Initialize cryptography on page load
74     initCrypto();
75 </script>
76 </body>
77 </html>
```

Listing 5: index.html - Frontend Client Application